Exercise 05: Malware and SQL Injection

Hiraku Morita

4 March 2026

In this exercise you will do the following:

- Construct and/or test out SQL injection strings to extract data from a MySQL server.
- Exploit bad local configuration & even worse policies.
- Experiment a bit with toy malware.

Please note that all questions (bullet points/paragraphs) marked with †. Please summarize your findings in a **report** and **submit** it on Itslearning. Although this is not mandatory, the material is related to the final exam.

Note: whenever you stumble upon a command you do not know yet, try calling it with the `-help` or `-h` parameter as `<command name> -help` or check its manual page `man <command name>`. In some cases, manual pages were replaced by the hypertext info system: `info <command name>`. And of course, a web search will surely also help getting a grasp of a manual or tutorial. See also the SQL tutorial at w3schools.

1 SQL Injection Preparation

Start up the Kali and the Metasploitable 3 VMs. Refresh on SQL injection, e.g., with the lecture slides or online sources like the w3schools SQL Tutorial.

From the assessment you performed with `nmap` and `gvm` in Exercise 02, notice that ports for TCP – Hypertext Transfer Protocol (HTTP) and a MySQL Server are open. What are the port numbers?

Unless stated, all the following actions are from Kali.

```
nmap -sV <METASPLOITABLE IP>
```

As there is a web server running, let's try it out. Try accessing the web server through a browser by just entering the IP address and port number:`<METASPLOITABLE IP>:80` or just `<METASPLOITABLE IP>` and hit enter. Notice the fact that we actually get a directory listing. Is this a well configured web server?

2 Spying with SQL Injection

SQL injection attacks are among the most common attacks. Given that data for dynamic web contents needs to be stored someplace, this shouldn't surprise, should it? In the following

part of the exercise, you will explore a classic SQL Injection vulnerability using DVWA (Damn Vulnerable Web Application) running on Metasploitable 2.

Let's go on by entering `<METASPLOITABLE IP>/dvwa` on a browser and log in with the default DVWA credentials:

- Username: admin
- Password: password

DVWA includes multiple security modes that intentionally change how vulnerable the application is. If the security level is set too high, SQL Injection will not work and the experiment will appear to "do nothing". In the left-hand navigation menu, click: DVWA Security, and in the Security Level dropdown, select:Low. Then, click the Submit button to apply the change.

From the left-hand menu, click on: SQL Injection. You will see a simple form asking for a User ID. Test the basic functionality Enter:1 and submit. The application should return the details of the user with ID 1 from the backend database.

Let's try the first SQL injection. Assuming that the text field's input is used directly in a SQL query, as the input is obviously not sanity-checked, try to end a likely quotation mark and use a logical condition to overwrite the likely `SELECT ...WHERE ...` query. Try to construct your input yourself, or try the following:

```
" OR 1=1
' OR 1=1
" OR 1=1#
' OR 1=1#
```

i.e., try combinations of three components:

1. one single or double quote, followed by,
2. a logical condition that will hold true, like `OR 1=1`.
3. Check if you need an extra character to make the whole thing work. For example a semicolon, to prevent execution of the rest of the statement, or a hash, can also do wonders.

What does the hash do? Will this generally work?

Questions †

† Please shortly discuss your opinion of this web server's configuration concerning directory listings.
† Please document the input values you tested (such as `' OR 1=1#` and other SQL injection payloads) and describe the results you observed.
† What type of SQLi attack works? Can you explain why?

Because of string concatenation and bad handling of user input, you should be able to see all users and their data. You want more? Let's try to get both usernames and passwords.

Nmap and Nessus revealed that there is a database on the server on port 3306. What service is running there? We now know the usernames. We can assume that there is a table with user information. Often such a table will simply be titled `users`. Here, we are making an

assumption, based on our experience of developing systems without bothering about security. :) But seriously, just giving the table another name is security by obscurity and will not replace real security measures.

Let's go on and try to get all usernames and passwords. Enter the following command into any of the fields:

```
' OR 1=1 UNION SELECT user, password FROM users #
```

In the above, the `UNION` enables you to execute two SQL statements. This is an example of a batch of SQL statements. Now scroll to the bottom of the webpage and you should see all the users and their passwords.

Questions †

† What do you notice about these passwords? Which hash algorithm is this?
† What is the # sign for? Can we generally assume it to do the trick?
† Include four relevant username/password combinations in your report. What is the issue with the passwords in the database and what could be done to secure them?

3 Elevation of Privilege

You have obtained user names and corresponding passwords. For going on, an interesting question is if we can elevate our privileges. A classical candidate for systems with shell access is `sudo`. There have been many security issues in `sudo` itself, but it is also very easy to mess up the configuration.

The simplest way though is to try if there are credentials which give you `sudo` and thus `root` access. Hint: the `sudo` configuration allows to limit use by groups. For many typical systems, you can check if a user can use default `sudo` root access by checking groups with the `groups` command. If the output contains `sudo` or `wheel` (depending on the system), you might have `sudo` access.

From your Kali machine, log in:

```
ssh msfadmin@<METASPLOITABLE IP>
```

If you get an error, try the fosslowing instead:

```
ssh -oHostKeyAlgorithms=+ssh-rsa -oPubkeyAcceptedAlgorithms=+ssh-rsa msfadmin@<
    METASPLOITABLE IP>
```

and put a password:

```
password: msfadmin
```

Now enter:

```
sudo -s
```

to get an interactive root shell, if available. If so, the user has root access and the command line prompt will change to:

```
root@metasploitable:~#
```

Note: on many systems, the last character of the prompt indicates if the prompt belongs to a normal user (e.g., `$`) or `root` (e.g., `#`). This simple exploit highlights the power of using SQL injection: root access with little effort. Of course, there were several underlying problems, which allowed to connect the vulnerability in the payroll PHP script to getting root access on the machine.

Questions †

- † Which are the individual issues that allowed us to go from a web interface to root access, and how would you address them as a server's operator to prevent them being exploited? Describe the issues you identified and try to come up with suggestions on how to fix them.
- † Can SQL Injection expose an otherwise inaccessible database server?
- † How likely do you think an attack scenario as presented here is?

4 Using our Foot in the Door for Access to Other Services

Stay in the `ssh` connection with root access. Now that you are on the server, let's find the `.php` file for that payroll website. To search, enter on the command line:

```
sudo find / -iname "*dvwa*"
```

You should get a list of locations with a file named `dvwa`.

```
/var/lib/mysql/dvwa
/var/www/dvwa/dvwa
/var/www/dvwa/dvwa/includes/dvwaPage.inc.php
/var/www/dvwa/dvwa/includes/dvwaPhpIds.inc.php
```

Questions †

- † Is `sudo` necessary? What do we gain by using it?
- † Are there other ways to search for a file? Which do you know?

Let's go to the directory:

```
cd /var/www/dvwa/
```

Let's analyze the PHP code and see if we can find interesting lines of code:

```
cat config/config.inc.php
```

Notice the connection string to the MySQL database? It's the very first PHP block:

```
$_DVWA[ 'db_server' ] = 'localhost';
$_DVWA[ 'db_database' ] = 'dvwa';
$_DVWA[ 'db_user' ] = 'root';
$_DVWA[ 'db_password' ] = '';
```

Questions †

- † Can you find anything interesting? Hint: it's trying to connect to the MySQL database. Check the documentation for the corresponding PHP functions.
- † What's the user name, password and database name?

Now you can easily access the database using that information and the `mysql` command, entering the password when prompted:

```
mysql -u root
```

Quick info about the above statement: It tries to connect to the MySQL server. `-h 127.0.0.1` specifies the database server with localhost's IP. `-P 3306` is the port number. `-u root` specifies root as the user name. If all goes well you will get the following prompt:

```
mysql>
```

Now the database is your oyster. :) You can add, edit, delete tables, columns, rows, etc. To get out of MySQL and return to the shell:

```
exit
```

Once inside, we can inspect all databases:

```
show databases;
use dvwa;
show tables;
select * from users;
```

This demonstrates how system compromise enables attackers to access backend databases more directly and completely than through SQL injection alone.

5 Reflection †

- † What was the problem with the web application?
- † Which ports and services were the problem associated with?
- † How did you exploit the vulnerability?
- † And what were you able to do?
- † How would you suggest to fix the problem? (Do some online research about SQL injection solutions.)
- † Draft, shortly and crisply, the relevant parts of a policy trying to prevent these issues.